

Requisitos atendidos pela aplicação (NTB Estoque)

Visão geral

Aplicação web em Laravel (PHP) para suporte a processos de **estoque** com integração ao **Omie**. Os principais módulos expostos no menu e rotas incluem: Notas Fiscais, Ordens de Produção, Transferências, Inventário, Produtos, Locais de Estoque, Lojas, Usuários e Logs de Integração.

Requisitos funcionais (RF)

RF-01 — Autenticação de usuários

- O sistema deve exigir login para acesso às áreas internas.
- **Evidência:** rotas protegidas por `auth` em `routes/web.php` ; controllers aplicam `$this->middleware('auth')` (ex.: `app/Http/Controllers/HomeController.php`).

RF-02 — Seleção de loja (contexto obrigatório)

- O usuário deve selecionar uma **loja atual** para operar módulos de negócio.
- Se não houver `current_loja_id` , o sistema deve bloquear o fluxo e redirecionar.
- **Evidência:** middleware `app/Http/Middleware/CheckCurrentLoja.php` ; rotas com `CheckCurrentLoja` em `routes/web.php` ; seletor de loja em `resources/views/layouts/menu.blade.php` .

RF-03 — Gestão de lojas (Admin)

- O sistema deve permitir CRUD de lojas e armazenar credenciais de integração Omie (App Key/Secret).
- **Evidência:** `app/Http/Controllers/LojaController.php` ; tabela `lojas` (campos `omie_app_key` , `omie_app_secret`) no schema.

RF-04 — Gestão de usuários (Admin)

- O sistema deve permitir listar, criar, editar e excluir usuários.
- O sistema deve permitir vincular usuário a lojas.
- **Evidência:** `app/Http/Controllers/User/UserController.php` ; tabela `users` e pivot `loja_user` .

RF-05 — Gestão de permissões por loja (Admin)

- O sistema deve suportar permissões por loja (usuário x permissão x loja).
- **Evidência:** `app/Services/CanService.php` ; controllers verificam `CanService::canPermissionLoja(...)` ; tabelas `permissaos` e `permissao_user` .
- **Catálogo base:** `app/Helpers/Constants.php` (`Constants::PERMISSOES`).

RF-06 — Consulta de Produtos

- O sistema deve permitir listar produtos por loja, com busca por descrição.
- **Evidência:** `app/Http/Controllers/ProdutoController.php` (rota `produto.index`).

RF-07 — Sincronização de Produtos com Omie

- O sistema deve permitir solicitar sincronização de produtos via Omie.
- **Evidência:** `app/Http/Controllers/ProdutoController.php` (rota `produto.update`), `app/Services/ProdutoService.php` (integração).

RF-08 — Consulta de Locais de Estoque

- O sistema deve permitir listar locais de estoque por loja, com filtro por descrição.
- **Evidência:** `app/Http/Controllers/LocalEstoqueController.php` (rota `locais-estoque.index`).

RF-09 — Sincronização de Locais de Estoque com Omie

- O sistema deve permitir solicitar sincronização de locais de estoque via Omie.
- **Evidência:** `app/Http/Controllers/LocalEstoqueController.php` (rota `locais-estoque.update`), `app/Services/LocalEstoqueService.php` .

RF-10 — Inventário: listar/filtrar por período e atributos

- O sistema deve permitir listar inventários por período e filtros (tipo/família).
- **Evidência:** `app/Http/Controllers/Inventario/InventarioController.php@index` ; rotas `inventario.index` .

RF-11 — Inventário: criar contagem

- O sistema deve permitir criar um inventário para um local de estoque, com data e motivo.
- O sistema deve impedir criação de novo inventário se já existir um "Em contagem" no mesmo local.
- **Evidência:** `InventarioController@store` + validações; tabela `inventarios` .

RF-12 — Inventário: registrar itens (contagem)

- O sistema deve permitir adicionar itens ao inventario por código do produto e quantidade.
- O sistema deve consultar/atualizar custo (CMC) via posição de estoque quando necessário.
- **Evidência:** `InventarioController@storeItem` ; `app/Services/PosicaoEstoqueService.php` ; tabela `inventario_items` .

RF-13 — Inventário: ajuste/integração Omie via fila

- O sistema deve processar ajustes no Omie para itens do inventário via job em background.
- Deve atualizar status do inventário e itens conforme processamento.
- **Evidência:** `InventarioController@finish` ; job `app/Jobs/InventarioJob.php` ; eventos de notificação.

RF-14 — Inventário: edição de quantidade e exclusão de itens

- O sistema deve permitir editar quantidade e excluir itens do inventário.
- **Evidência:** rotas `inventario.setQuantidade` , `inventario.editQuantidade` , `inventarioitem.destroy` em `routes/web.php` ; métodos correspondentes em `InventarioController` .

RF-15 — Inventário: duplicação

- O sistema deve permitir duplicar um inventário existente.
- **Evidência:** rota `inventario.duplicar` e método `InventarioController@duplicar` .

RF-16 — Inventário: geração de PDF

- O sistema deve permitir gerar PDF do inventário.
- **Evidência:** `InventarioController@pdf` .

RF-17 — Transferências: listar/filtrar por período e atributos

- O sistema deve permitir listar transferências por período e filtros (tipo/família).
- **Evidência:** `app/Http/Controllers/Transferencia/TransfersController.php@index` ; rotas `transfers.index` .

RF-18 — Transferências: criar transferência

- O sistema deve permitir criar transferência entre local origem e destino, com data e motivo.
- Deve impedir criar nova transferência se houver outra "Processando" para o mesmo par origem/destino e data.
- **Evidência:** `TransfersController@store` ; tabela `transferencias` .

RF-19 — Transferências: registrar itens

- O sistema deve permitir adicionar itens (movimentos) na transferência via código do produto e quantidade.
- Deve tratar “produto não encontrado”.
- Deve obter CMC via posição de estoque quando necessário.
- **Evidência:** `TransfersController@storeItem` ; model `app/Models/Movimento.php` ; `PosicaoEstoqueService` .

RF-20 — Transferências: finalizar e processar no Omie via fila

- O sistema deve permitir finalizar uma transferência e processar ajustes no Omie via job assíncrono.
- Deve notificar usuário sobre progresso e conclusão.
- **Evidência:** `TransfersController@finish` ; job `app/Jobs/TransferJob.php` ; eventos `NotificaUserEvent` .

RF-21 — Transferências: editar quantidade e reprocessar

- O sistema deve permitir editar quantidade de um movimento e reprocessar no Omie.
- **Evidência:** `TransfersController@editQuantidade` ; job `TransferJob` .

RF-22 — Transferências: excluir item e excluir transferência

- O sistema deve permitir excluir item e excluir transferência (com exclusão dos movimentos).
- Deve tentar excluir o ajuste correspondente no Omie quando aplicável.
- **Evidência:** `TransfersController@destroyItem` , `TransfersController@destroy` ; `app/Services/OmieService.php` .

RF-23 — Transferências: duplicação

- O sistema deve permitir duplicar uma transferência, replicando movimentos.
- **Evidência:** rota `transfers.duplicar` ; `TransfersController@duplicar` .

RF-24 — Transferências: geração de PDF

- O sistema deve permitir gerar PDF da transferência.
- **Evidência:** rota `transfers.pdf` ; `TransfersController@pdf` ; view `resources/views/transfers/pdf.blade.php` .

RF-25 — Notas Fiscais: listagem e filtros

- O sistema deve permitir listar notas fiscais por período e filtros (número, fornecedor, produto, tipo e status).
- Deve carregar itens e produto associado.
- **Evidência:** `app/Http/Controllers/NotaFiscal/NotafiscalController.php@index` ; tabelas

RF-26 — Notas Fiscais: visualizar itens e ajustar quantidade local

- O sistema deve permitir visualizar itens de uma nota fiscal e registrar quantidade localmente.
- **Evidência:** `NotafiscalController@itens` , `NotafiscalController@setQuantidade` .

RF-27 — Notas Fiscais: impressão de etiquetas em PDF

- O sistema deve gerar etiquetas em PDF para itens da NFe.
- **Evidência:** `NotafiscalController@imprimir` ; `view resources/views/etiqueta/imprimir.blade.php` ; geração via PDF (Snappy).

RF-28 — Notas Fiscais: sincronização via Omie

- O sistema deve solicitar sincronização de notas fiscais via Omie.
- **Evidência:** `NotafiscalController@syncNotasFiscais` ; `app/Services/NotaFiscalService.php` .

RF-29 — Ordens de Produção: listagem e filtros

- O sistema deve permitir listar OPs por período e filtros (número, tipo, produto, concluído).
- **Evidência:** `app/Http/Controllers/OrdemProducao/OrdemProducaoController.php@index` ; tabela `ordem_producao` .

RF-30 — Ordens de Produção: atualizar validade e quantidade

- O sistema deve permitir registrar validade e quantidade em uma OP.
- **Evidência:** `OrdemProducaoController@setValidade` , `OrdemProducaoController@setQuantidade` .

RF-31 — Ordens de Produção: impressão de etiquetas em PDF

- O sistema deve gerar etiquetas em PDF para OP.
- **Evidência:** `OrdemProducaoController@imprimir` .

RF-32 — Ordens de Produção: sincronização via Omie

- O sistema deve solicitar sincronização de OPs via Omie.
- **Evidência:** `OrdemProducaoController@syncOrdensProducao` ; `app/Services/OrdemProducaoService.php` .

RF-33 — Webhook Omie (API)

- O sistema deve receber eventos do Omie via endpoint HTTP e persistir a mensagem.
- Deve disparar processamento automático quando solicitado.

- Deve disparar processamento assíncrono quando aplicável.
- **Evidência:** rota `POST /api/webhook` em `routes/api.php`; controller `app/Http/Controllers/API/OmieWebhookController.php`; job `app/Jobs/OmieWebhookJob.php`; model `app/Models/Webhook.php`.

RF-34 — Processamento por eventos (Omie Webhook)

- Ao salvar webhook, o sistema deve reagir a tópicos conhecidos (Produto, LocalEstoque, RecebimentoProduto, OrdemProducao).
- **Evidência:** hook `created()` em `app/Models/Webhook.php` chamando services correspondentes.

RF-35 — Logs de integração (auditoria técnica)

- O sistema deve registrar tentativas de integração (request/response/status) e permitir consulta via tela.
- **Evidência:** tabela `integration_attempts`; trait `app/Services/IntegrationAttemptsTrait.php`; `app/Http/Controllers/IntegrationAttemptController.php`; rota `log.index`.

Requisitos não funcionais (RNF)

RNF-01 — Plataforma e stack

- **Backend:** PHP 8.4+ e Laravel 12.
- **Banco:** MySQL (com filas via `jobs` em database).
- **Frontend:** Vite + JS; TailwindCSS; Bootstrap (uso em views).
- **Evidência:** `composer.json`, `config/queue.php`, `vite.config.js`, `resources/js/*`.

RNF-02 — Segurança: autenticação e controle de acesso

- Deve restringir acesso por autenticação (`auth`) e por perfil/permissões.
- Deve impedir ações de Admin por usuários não-admin.
- **Evidência:** middleware `auth`; checks `auth()->user()->perfil !== 'Admin'`; `CanService::canPermissionLoja`.

RNF-03 — Multiloja (isolamento lógico)

- Operações de negócio devem ser filtradas por `loja_id` (loja atual do usuário).
- **Evidência:** queries em controllers usam `where('loja_id', auth()->user()->current_loja_id)`; middleware `CheckCurrentLoja`.

RNF-04 — Processamento assíncrono e filas

- Processos de integração Omie devem ser executados em background (fila database), com

múltiplas filas por domínio (`notafiscal` , `ordemproducao` , `posicaoestoque` , `produto` , `notifications` , `default`).

- **Evidência:** `config/queue.php` ; scripts `composer.json` (`composer run queues`); jobs em `app/Jobs/*` .

RNF-05 — Notificações em tempo real (WebSocket)

- O sistema deve notificar usuários via broadcasting (Reverb) em canais privados e canal global.
- **Evidência:** eventos `app/Events/NotificaUserEvent.php` , `app/Events/NotificaAllEvent.php` ; canais em `routes/channels.php` ; cliente Echo em `resources/js/echo.js` ; `config/broadcasting.php` .

RNF-06 — Observabilidade: logs de aplicação e logs de integração

- Deve registrar falhas e avisos relevantes em logs.
- Deve registrar tentativas de integração com request/response/status em `integration_attempts` .
- **Evidência:** uso de `Log::*` em `controllers/services/jobs`; `IntegrationAttemptsTrait` .

RNF-07 — Confiabilidade: reprocessamento e estados

- Processos longos devem manter estado (ex.: status em `lojas` e `transferencias / inventarios`) e permitir acompanhar conclusão.
- **Evidência:** campos `*_status` e `*_ultima_atualizacao` em `lojas` ; status em `inventarios` , `inventario_items` , `transferencias` , `movimentos` .

RNF-08 — Rate limit Omie

- Chamadas ao Omie devem respeitar rate limit e concorrência conforme regras informadas (por IP, por IP+AppKey+Método, simultâneas e métodos especiais).
- **Evidência:** `app/Services/OmieService.php` (controle via `Cache` , `Cache::lock` , contadores e espera).
- **Observação:** o rate limit do Omie também é tratado por alguns serviços/Jobs via `sleep()` quando a API retorna sinalização (ex.: status 425 em `NotaFiscalService`).

RNF-09 — Geração de PDFs

- O sistema deve gerar PDFs (relatórios/etiquetas) no servidor.
- **Evidência:** `barryvdh/laravel-snappy` em `composer.json` ; uso de `PDF/Pdf::loadView(...)` em controllers.

RNF-10 — Internacionalização/idioma

- O sistema deve operar com textos em PT-BR.